

SECURITY

HORIZON GRID

Security Documentation

Version: 0.2.3

Documentation prepared: 2026-08-20

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID Security Reference	3
---------------------------------	---

HORIZON GRID Security Reference

Purpose and Scope

This is a standalone security reference for HORIZON GRID, intended to be read on its own by a security reviewer, auditor, or prospective operator who does not have time to read the full documentation set. It pulls together, in one place, the controls that matter most when deciding whether to trust this platform with real indicators and real credentials: authentication, authorization, credential handling, input validation, outbound-request safety, export safety, the AI's relationship to the numbers it narrates, rate limiting, and audit logging -- plus an honest list of what is *not* yet in place.

Every claim below is traceable to a specific source file in `backend/app/`, confirmed by direct inspection, not inferred from a docstring or a design intention. Three companion chapters in the full documentation set go deeper on individual subsystems and are referenced inline rather than repeated: the **Backend Security Architecture** chapter (JWT/RBAC/secrets/audit implementation detail, line-numbered), the **Security Architecture** technical appendix (network exposure, CORS), and the **Security and Data Handling** user-facing chapter (the same material explained for a non-engineer). This document's job is different from all three: it is the one file a reader can hand to someone outside the project and have it make sense without any other context.

1. Authentication

Sign-in is JWT-based (`app/auth/security.py` , HS256 via `python-jose`), with passwords hashed by `passlib` 's `bcrypt` scheme -- a plaintext password is never persisted, only its hash. Two token types are minted at login: a 30-minute access token and a 7-day refresh token, each carrying `sub` (email), `role` , `token_version` , `type` , `iat` , and `exp` claims. `get_current_user` (`app/auth/rbac.py`), the dependency every protected route shares, only accepts a token whose `type` claim is "access" , and it re-reads `is_active` from the database on every single request rather than trusting anything cached in the token -- so disabling an account takes effect on that account's very next click, not whenever its token happens to expire.

Instant revocation on password reset, keyed by `token_version` . A stateless JWT's usual weakness is that an already-issued token stays valid under the *old* password until it naturally expires, because nothing forces the holder to re-authenticate. HORIZON GRID closes this specific gap with a `token_version` integer column on `users` , echoed into every token's `token_version` claim at mint time. `get_current_user` rejects any token whose embedded value no longer matches the current database value:

```
# app/auth/rbac.py
if payload.get("token_version", 0) != user.token_version:
    raise credentials_error
```

An administrator resetting a user's password increments that user's `token_version` , which instantly invalidates every access and refresh token that user already holds, everywhere, without needing a separate revocation table or a Redis denylist -- the very next request with an old token simply fails the equality check above and the caller is forced to sign in again with the new password. `payload.get(..., 0)` treats a token minted before this feature existed (no claim at all) as version `0` , which matches every existing user row's initial column value, so introducing this control never force-logged-out anyone who wasn't supposed to be. The same mechanism fires on a role change or an account disable/enable -- both take effect immediately because `get_current_user` re-reads `role` and `is_active` from the database on every request in the first place, needing no new machinery at all.

Bootstrap and account creation. The very first user ever created on an instance is automatically granted `ADMIN` ; every registration attempt after that is rejected outright (`403` , "Self-registration is closed"). There is no seed script and no default account -- this

bootstrap rule, exercised by the Windows Setup Wizard at install time, is the platform's only path to an initial administrator. Every account after that first one is created by an existing administrator from the Administration page, who picks its role up front.

What is honestly not yet in place: no self-service "log out everywhere" (only an admin-driven password reset triggers revocation -- a user cannot invalidate their own other sessions without changing their password), no rate limiting on `/auth/register`, no account lockout on either endpoint, and no multi-factor authentication. `/auth/login` itself did gain a real per-account rate limiter in a later mission-critical-reliability review (v0.2.3) -- see §12. None of the remaining gaps are silently glossed over either.

2. Authorization: Role-Based Access Control

Exactly three roles exist -- `ADMIN`, `ANALYST`, `VIEWER` (`app/models/user.py`'s `Role` enum) -- resolved through a single flat permission-string matrix, `ROLE_PERMISSIONS`, checked by one shared dependency, `require_permission(permission: str)` (`app/auth/rbac.py`). A route either declares a required permission string or it enforces none; there is no separate ad hoc authorization logic scattered through the codebase for any route this document covers.

Permission	ADMIN	ANALYST	VIEWER
<code>lookup:create</code>	yes	yes	
<code>lookup:read</code>	yes	yes	yes
<code>lookup:export</code>	yes	yes	
<code>evidence:read</code>	yes	yes	yes
<code>analysis:generate</code>	yes	yes	
<code>copilot:query</code>	yes	yes	
<code>hunting:generate</code>	yes	yes	
<code>basket:manage</code>	yes	yes	
<code>case:create</code> / <code>case:write</code> / <code>case:close</code>	yes	yes	
<code>case:read</code>	yes	yes	yes
<code>security_assessment:create</code>	yes	yes	
<code>security_assessment:read</code>	yes	yes	yes
<code>dashboard:read</code>	yes	yes	yes
<code>provider:manage</code>	yes		
<code>user:manage</code>	yes		
<code>audit:read</code>	yes		

`VIEWER` is provably read-only: scanning its row above, there is no create, write, manage, export, or generate permission anywhere in it. A `VIEWER` account can look at investigations, evidence, cases, security-assessment findings, and dashboard KPIs, and can do nothing else -- it cannot launch a new investigation, cannot export a file, and cannot trigger a security scan. `dashboard:read` is deliberately the one permission granted to all three roles without restriction: it gates read-only KPI/executive-summary numbers, and was confirmed during release QA to expose no credential, no per-user field, and no otherwise-gated data to a `VIEWER` who holds it.

A permission failure raises `HTTP 403` naming both the caller's own role and the missing permission string -- convenient for debugging, and not a material information leak, since the caller already knows its own role from its own JWT.

Multi-admin management is race-safe. Any existing administrator can create another administrator; there is no single-admin bottleneck and no hardcoded admin account. Two protections stop an administrator action from leaving the platform with zero administrators: a self-role-change is rejected outright (an admin cannot demote themselves -- another admin has to do it), and the last-administrator check takes a `SELECT ... FOR UPDATE` lock across **every** `ADMIN`-role row before counting how many would remain active, specifically closing a real race where two concurrent requests could each disable a *different* one of the last two administrators and both succeed, leaving zero. There is no hard delete for user accounts -- case/evidence/basket foreign keys require it, and it would otherwise be possible to silently orphan an audit trail's own actor references -- so disabling an account is the only removal path, and disabling always leaves at least one active administrator standing.

A dedicated cross-cutting RBAC sweep performed during the most recent release cycle checked every mission-touched route against `ROLE_PERMISSIONS` end to end (route to dependency to permission string to role table), independently re-verified by a second adversarial pass. **Result: clean** -- no permission-string mismatch, no role silently missing an entry, no route granting wider or narrower access than the matrix intends. Full implementation detail (line references, the complete route-by-route enforcement table) is in the Backend Security Architecture chapter; that chapter also discloses the one currently unenforced permission (`lookup:export`) was, until the fix described in §6 below, defined but not wired to its intended route) and the one deliberate scope choice (cases and lookups are team-visible to any role holding the relevant permission, not restricted per-owner -- there is no per-case ACL).

3. Credential Storage

Two credential paths exist. `.env`-based settings (provider/AI keys set at container start) are frozen for the process lifetime and require a restart to change. The path that matters for this section is the newer one: **runtime-configured** provider and AI-backend credentials, entered live through the Manage Providers UI with no restart required.

Encrypted at rest. Every runtime-configured credential is Fernet-encrypted (`app/core/crypto.py`) before it is ever written to the database; the `provider_runtime_configs.encrypted_credentials` column holds ciphertext only. The Fernet key itself comes from an explicit `encryption_master_key` setting if one is configured, or -- by default -- is deterministically derived via HKDF-SHA256 from the platform's existing `jwt_secret_key`, domain-separated by a fixed info string. That fallback is a documented tradeoff, not an oversight: it means every existing install already has a working encryption key with zero migration effort, at the cost of key independence -- compromising `jwt_secret_key` also exposes the derived Fernet key. An operator wanting genuine key separation sets `encryption_master_key` explicitly. `decrypt_secret()` returns an empty string on any decryption failure (a corrupted row, a rotated key) rather than raising, so a broken credential degrades to "not configured," never a `500`.

Masked, never round-tripped. Every API response that surfaces a runtime-configured credential -- `GET /runtime/ai-providers`, `GET /runtime/ioc-providers` -- decrypts internally only to immediately re-mask it before the response leaves the process:

```
# app/core/crypto.py
def mask_secret(plaintext: str, visible_suffix: int = 4) -> str:
    """sk-abc123xyz" -> "*****3xyz" for display -- never round-trippable
    back to the real value, unlike returning a truncated real prefix."""
```

`mask_secret()` replaces every character except the last four with `*`, and -- critically -- there is no function anywhere in this codebase that reverses it. No route ever returns a decrypted runtime-configured credential to a browser.

Why "Test Connection" requires retyping an already-saved key. This is deliberate, not a UI oversight. `POST /providers/{provider_id}/test` and `POST /ai/test` both take the *candidate credentials to test* directly in the request body (`ProviderTestRequest.credentials: dict[str, str]`, `AITestRequest.credentials: dict[str, str]`) and make one real outbound call with exactly that value -- both routes' own docstrings state the credentials passed in are "never persisted, never read from settings." Nothing in the backend reads a previously-saved credential back out to test it, because there is no reversible representation of a saved credential anywhere to read: the database only ever holds ciphertext, and the API only ever hands back a masked string. If the frontend pre-filled a masked value like `*****3xyz` and submitted that as the "test," it would either fail meaninglessly against the real provider or -- worse -- give a false sense that the *saved* key was re-validated when nothing of the sort

happened. Requiring the operator to type the real value again for a test is the direct, correct consequence of the masking and non-persistence guarantees above holding without exception, not a gap in convenience the UI simply hasn't gotten around to closing. This was confirmed, during the most recent release's QA cycle, to be a real point of confusion for a real user encountering it for the first time -- which is exactly why it is called out explicitly here rather than left to be rediscovered by surprise.

On Windows installs, the legacy `.env` path is hardened at the filesystem level: `jwt_secret_key` and the datastore passwords are generated with .NET's cryptographically secure `RandomNumberGenerator` (not PowerShell's `Get-Random`), and the written file's NTFS ACL is immediately restricted to Administrators and SYSTEM only via `icacls`.

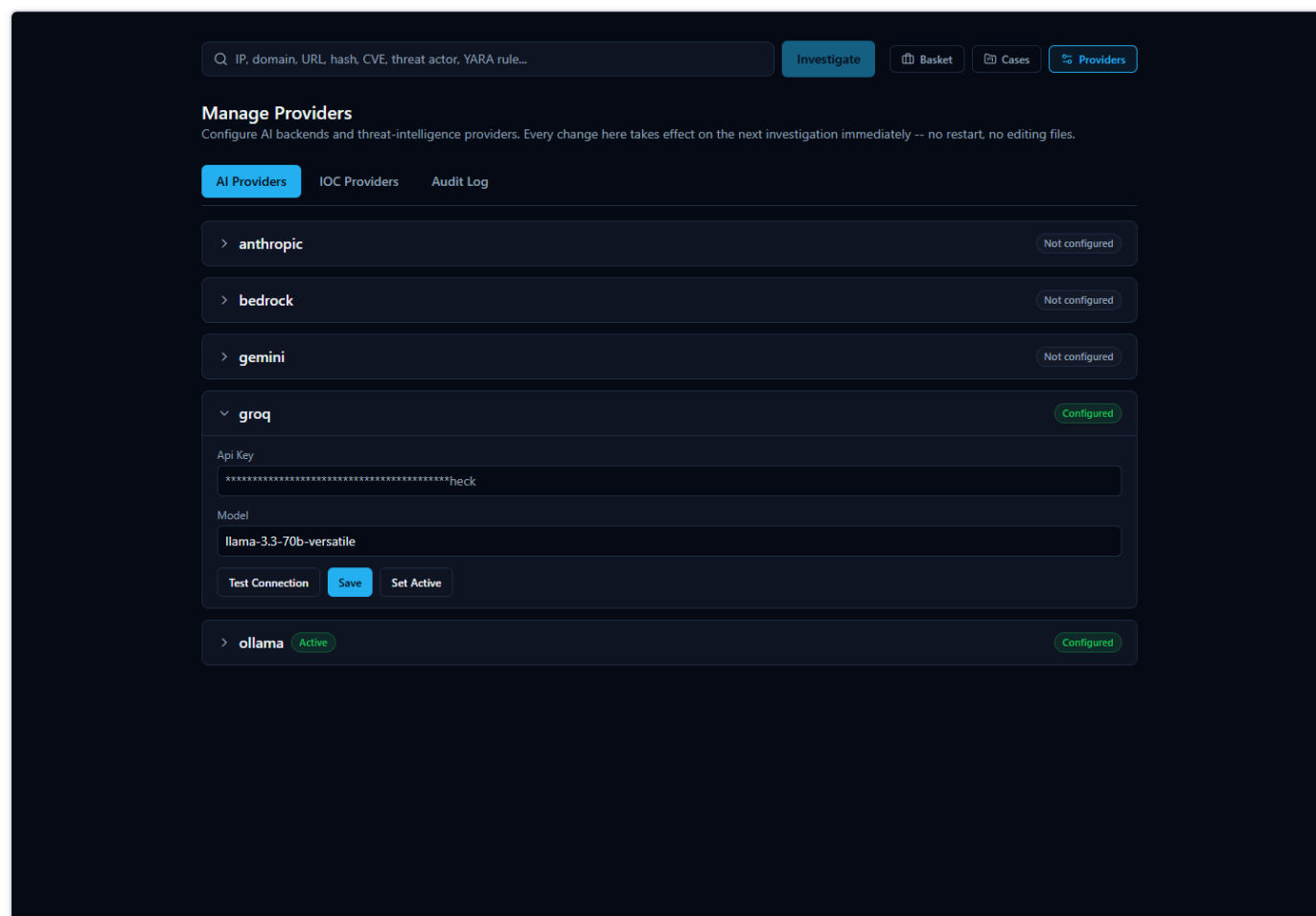


Figure 1 — A runtime-configured provider's credential field showing only a masked value (e.g. "*****3xyz") in the Manage Providers UI -- the real key is never returned by any API response.

4. Input Validation and IOC Handling

`LookupCreateRequest.value` -- the raw string a user submits for investigation -- is an unconstrained string on the server side. IOC-type detection (`app/ioc/detector.py`) is regex-based *classification* (deciding whether a string looks like an IPv4 address, a SHA256 hash, a domain, a CVE ID, and so on), not sanitization: it exists to route a value to the right providers, not to reject or clean dangerous input. This is a load-bearing design fact worth stating plainly, because it means downstream consumers of a raw IOC value -- exports, the AI prompt, provider connectors -- are each individually responsible for treating that value as untrusted, and this document's remaining sections (§6, §7) are exactly the record of where that responsibility was and wasn't discharged correctly. Registration's `RegisterRequest.password` has a server-enforced 8-character minimum (and a 72-byte maximum, matching bcrypt's own effective limit) but no complexity rule beyond length.

5. Server-Side Request Forgery (SSRF) Protection

One place in this codebase makes a server-side HTTP call to a host fully chosen by an operator, rather than a fixed provider domain: the Ollama `base_url` field, since a local AI backend can legitimately point at any host on the operator's own network.

`app/core/url_safety.py`'s `assert_safe_outbound_url()` is the guard applied before that URL is ever fetched:

```
# app/core/url_safety.py
def assert_safe_outbound_url(url: str) -> None:
    """Raises ValueError if `url` is not safe to fetch server-side."""
    parsed = urlparse(url)
    if parsed.scheme not in ("http", "https"):
        raise ValueError(...)
    ...
    for addr in addrs:
        ip = ipaddress.ip_address(addr)
        if ip.is_link_local:
            raise ValueError(
                f"Host {parsed.hostname!r} resolves to a link-local address ({addr}), "
                "which includes cloud instance-metadata services -- refusing to connect."
            )
```

Two design choices here are deliberate, not accidental gaps. First, only `http / https` schemes are permitted at all -- rejecting `file://`, `gopher://`, and every other scheme outright. Second, the function resolves the hostname and inspects every returned address rather than pattern-matching the URL string, which is what actually closes a DNS-rebinding-style bypass (a hostname that resolves differently at check time versus fetch time cannot be reasoned about safely from the string alone). Third, and most deliberately: it does **not** block loopback or RFC 1918 private-IP ranges. That is not an oversight -- Ollama's entire legitimate use case is a local or LAN model server (this platform's own default `OLLAMA_BASE_URL` points at `host.docker.internal`), so blocking private ranges would break real, intended functionality rather than stop a real attack. What it specifically blocks is link-local address space (`169.254.0.0/16`, `fe80::/10`), which has no legitimate Ollama use case and is where essentially every major cloud provider's instance-metadata service lives (`169.254.169.254`) -- the single highest-value SSRF target this particular code path could otherwise be tricked into reaching. This is a narrowly scoped, honestly-reasoned control for the one outbound-URL surface that exists today, not a generic egress firewall.

6. Export Security: CSV and PDF Injection Protections

A user-supplied IOC value or AI-generated assessment text flows, unmodified, into two file-export formats server-side: CSV and PDF (`app/api/routes/lookup.py`). Both formats were found, during the most recent release's own adversarial QA pass, to have real injection vulnerabilities, and both were fixed and regression-tested before release.

CSV / formula injection (CWE-1236). A cell whose text begins with `=`, `+`, `-`, or `@` is interpreted as a live formula by Excel or LibreOffice when the exported file is reopened -- a free-text IOC value of `=1+1+cmd|calc!A1` was confirmed, live, to produce exactly that raw, unescaped payload in an exported CSV before the fix. `csv.writer`'s own quoting does not defend against this: `QUOTE_MINIMAL` leaves a comma-free formula completely unquoted, and even `QUOTE_ALL` only wraps it in quotes the spreadsheet application strips back out during its own parsing, after which the remaining text still begins with `=`. The fix, `_csv_safe()`, prefixes a literal leading apostrophe onto any string field beginning with one of those four characters -- the standard mitigation, since the apostrophe survives CSV parsing and forces the cell to render as literal text. It is applied to every field that can ever contain free text influenced by an outside party (the IOC value itself, provider name, source URL, error message); fields drawn from fixed enums or numeric columns are never free text and are left untouched.

PDF / markup injection and crash. ReportLab's `Paragraph` object parses its input as a small XML/HTML-like markup dialect, not literal text. Before the fix, IOC values and AI-generated assessment text were interpolated directly into `Paragraph()` calls with zero

escaping, which was confirmed, live, to produce two distinct failure modes: an IOC value containing a stray unclosed tag (`<tag AAAA`) crashed the export with an uncaught `ValueError` and an HTTP 500 -- a *permanent* failure for that lookup's PDF export, since the same malformed value would be re-interpolated on every retry -- and a well-formed tag (`<font color="red" size="40">FAKE-VERDICT-INJECTED</font>`) was silently accepted as real formatting, letting attacker- or AI-influenced text inject spoofed styling into what looks like an official report. The fix, `_pdf_esc()`, XML-escapes every dynamic value (`&`, `<`, `>`) before it reaches a `Paragraph()` call, which fixes both failure modes at once: a malformed tag can no longer reach the markup parser as a tag at all, and a well-formed-looking tag renders as visible literal text instead of being applied as formatting. `Preformatted` blocks (used for AI-generated detection-rule bodies) were confirmed, by reading ReportLab's own implementation, to never invoke the markup parser in the first place, so they are intentionally left unescaped rather than double-processed.

Export permission gate. A related, independent bug existed alongside the two above: the export endpoint was gated on `lookup:read` rather than the dedicated `lookup:export` permission already defined in the role matrix -- meaning a `VIEWER` account, which holds `lookup:read` but never `lookup:export`, could successfully download a real PDF/CSV file despite the permission matrix explicitly withholding export from that role. This has been fixed; `export_lookup()` now depends on `require_permission("lookup:export")`, matching §2's table exactly.

All three fixes are confirmed present in the current codebase and regression-tested, not merely proposed.

7. AI Prompt-Injection Posture: Why the AI Cannot Set Its Own Score

The single most important security property of the AI layer is architectural, not a prompt instruction: **the risk numbers are computed before the AI is ever called, from a deterministic, non-AI scoring engine** (`app/scoring/engine.py`, `SCORING_ENGINE_VERSION="1.0"`), and the AI is given those numbers as a fixed fact it can only narrate, never set.

This exists specifically because prompt wording alone cannot reliably stop a language model -- especially a small local model -- from overriding a field it is told not to touch. That is not a hypothetical concern for this platform: a real, reproduced failure is documented in the codebase's own comments -- looking up the real MD5 hash of the EICAR antivirus test file against zero configured providers (i.e., with no actual evidence at all) still caused the default local model to return `final_verdict="highly_malicious"` and `malicious_probability=92`, fabricating an "association with ransomware and trojans" purely from its own pretrained knowledge, in direct violation of its own system-prompt instruction never to fabricate. The lesson taken from that incident is applied everywhere the AI layer touches a security-relevant number: decide the number deterministically first, then tell the AI the decided number as a given fact and give it a narrower job -- write a verdict and rationale *consistent with* the number, never invent one.

Concretely, in `generate_final_assessment()` (`app/ai/service.py`):

- If there is no provider data and no correlation evidence at all, the AI is **not called**. A result built directly from the scoring engine's own output (`final_verdict=UNKNOWN`, with risk fields taken from the scoring engine, which itself produces an honest all-zero result when there is truly zero evidence) is returned instead. This is recorded as `ai_outcome="skipped_no_evidence"` -- a *correct decision not to call the AI*, distinct from a failure.
- When there is evidence, the deterministic score is embedded directly in the prompt under a heading that says outright: `"Deterministic risk assessment (already computed -- GIVEN, do not recalculate)"`, and the model is instructed to "echo the given ... values verbatim into the risk object."
- Even if the model ignores that instruction -- and nothing besides the instruction itself stops it from doing so -- the code path that follows unconditionally overwrites `overall_risk_score`, `confidence_score`, `malicious_probability`, and `severity` with the scoring engine's real values before the assessment is ever returned or persisted. This is the exact same mechanical-overwrite pattern already used elsewhere in this codebase for `ai_backend` / `ai_model` (fields the model is told not to fill in, and which are overwritten regardless).
- Critically, that overwrite is not a blind field swap: the *entire* `FinalAssessment` object is re-validated (`model_validate()`) with the real deterministic risk spliced in, re-running every one of the model's own consistency validators -- including the one that rejects a verdict like `"malicious"` paired with a `malicious_probability` under 30 -- against the values that will *actually* be persisted, not just the model's own self-consistent-but-wrong pair. A model that emits an internally consistent but factually wrong result no

longer sails through unchecked; if its verdict doesn't survive contact with the real numbers, the request retries once with the same given numbers, giving the model a second chance to choose a verdict consistent with the ground truth it was handed.

- If AI generation fails outright after retrying, the fallback assessment still carries the real deterministic score (never hardcoded zeros) with `final_verdict=UNKNOWN` and `ai_outcome="failed"` -- a genuine failure, recorded honestly as one, but the number itself is never lost because it never depended on the AI succeeding in the first place.

The practical consequence for a crafted or adversarial IOC value: there is no prompt an attacker can smuggle into an IOC string, a provider's free-text field, or anywhere else in the pipeline that changes `overall_risk_score`, `confidence_score`, `malicious_probability`, or `severity`, because none of those four fields are ever taken from the AI's own output regardless of what it produces. The AI's only real degree of freedom is prose and verdict *choice among options the numbers already constrain* -- and re-running the platform's own analysis against a different AI backend on the same evidence produces the identical score every time, differing only in wording.

Two related, code-level guards close adjacent gaps in the same spirit: a grounding pass strips any AI-cited MITRE ATT&CK technique or agreeing/disagreeing-provider claim that isn't backed by a real correlation edge or provider record, and a citation-stripping step on every analyst-facing explanation endpoint (the "Why?", "Challenge This Verdict", and Copilot Q&A tabs) removes any evidence-ID citation that doesn't match a real evidence item for that lookup, never inventing a replacement. Full detail on both lives in the AI Architecture technical appendix; they are noted here because they follow the identical philosophy as the score-overwrite above -- verify the AI's output against real data, mechanically, rather than trust the instruction that asked for it to be accurate.

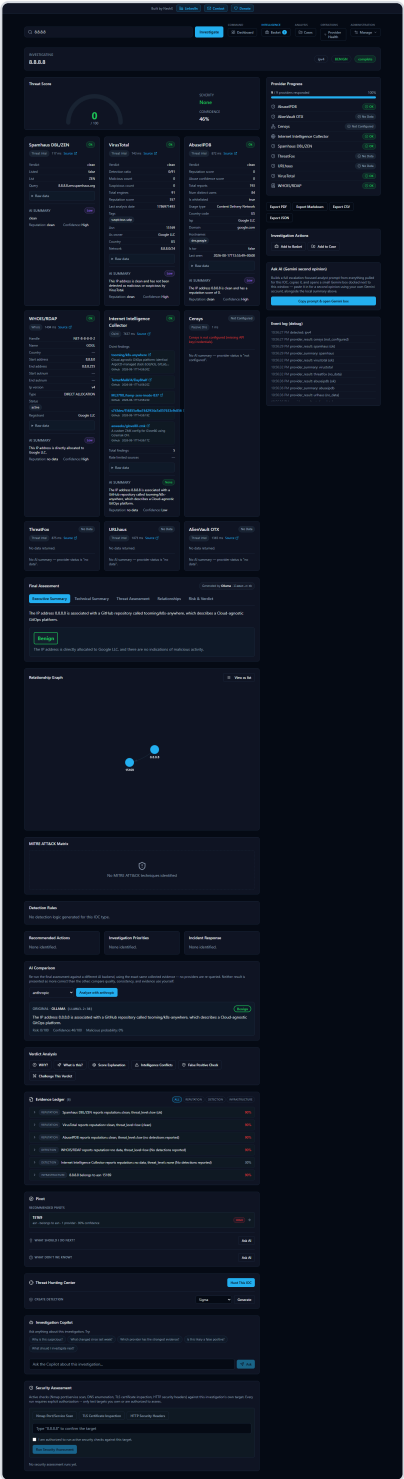


Figure 2 — Diagram: the deterministic scoring engine computes overall_risk_score/confidence_score/malicious_probability/severity from provider and correlation evidence BEFORE any AI call; the AI receives those numbers as a fixed input and the values are mechanically re-applied to its output regardless of what it produces.

8. Scoring-Engine Integrity: The Correlation-Flood Fix

A deterministic score is only a real security property if the *inputs* to that determinism can't themselves be gamed. During the most recent release's adversarial red-team pass, exactly that gap was found and fixed in the correlation half of the scoring engine.

The provider-verdict half of the engine already had an anti-flood defense: a single provider's "malicious" flag, with no other provider agreeing or disagreeing, is capped at 40% of that component's maximum strength (`_corroboration_factor(1) == 0.40`); it takes five independently agreeing providers to reach full strength. The correlation-graph half -- which scores malware/threat-actor/campaign associations, MITRE technique usage, and CVE-exploitation relationships discovered in the evidence graph -- had no equivalent defense until this fix. The vulnerability: a single free, unprivileged community account on OTX, ThreatFox, or MalwareBazaar can list several distinct free-text malware-family or threat-actor names against one indicator inside a single pulse or submission. Because those are *distinct* values from *one* provider, the correlation engine creates a separate graph edge for each one rather than merging them into a single corroborated edge the way an identical claim from two different providers would be merged -- meaning one unprivileged, no-cost account could single-handedly saturate the correlation component and push an indicator's score into the "high" severity band, with zero real infrastructure and zero genuine cross-provider corroboration behind it. This directly contradicted the scoring engine's own "conservative by construction" guarantee, and was numerically reproduced by two independent reviewers before being fixed.

The fix applies the identical corroboration-discount philosophy already used for provider votes, now keyed on the number of **distinct providers** asserting any qualifying correlation edge (not the number of edges):

```
# app/scoring/engine.py
def _correlation_fraction(edges):
    qualifying = [edge for edge in edges if edge.relationship in _QUALIFYING_RELATIONSHIPS]
    total_confidence = sum(edge.confidence for edge in qualifying)
    distinct_providers = {p for edge in qualifying for p in edge.provenance.split(",")}
    corroboration = _corroboration_factor(len(distinct_providers))
    return _clip(100.0 * total_confidence * corroboration / _CORRELATION_SATURATION) / 100.0
```

A single provider's edges -- however many distinct fabricated-looking values they contain -- are now capped at 40% of the correlation component's strength, identical to a lone provider vote. Genuine corroboration from two or more independent providers scales back up toward full strength, and is deliberately **not** penalized by this fix -- the goal is closing the "one source, many distinct unmerged claims" flood, not punishing real multi-provider agreement, which already carried its own confidence boost from the correlation engine before this component ever sees it.

This fix was unit-tested (22/22 scoring-engine tests passing, including a new regression test proving the fix discriminates a genuine flood from real multi-provider corroboration) and independently code-reviewed twice before its live-container re-verification against a real running instance -- disclosed at one point as still pending, honestly, rather than silently assumed complete, because the installed copy's containers were temporarily down for an unrelated installer test. That installer test has since completed and the containers came back up on a genuinely fresh install; the anti-flood fix has now been re-confirmed live on that fresh install (22/22 scoring-engine tests, full regression suite 308/0 failed).

One further, smaller item disclosed in the same review: `_provider_votes()` would treat a `NaN` / `Infinity` value in a provider's own detection-ratio fields as full-strength "malicious" rather than rejecting it outright. No live provider today actually produces such a value -- VirusTotal, the only connector populating those fields, derives them from its own server-computed statistics, not from attacker-controllable text -- so this is recorded as a non-blocking hardening recommendation, not an active exploit path.

9. Rate Limiting

Exactly one rate limiter exists anywhere in this codebase: a Redis fixed-window limiter, keyed per authenticated user, applied solely to `POST /lookup/stream` -- the endpoint that launches a new investigation:

```
# app/api/routes/lookup.py
limiter = RateLimiter(
    f"lookup_create:{user.id}",
    max_calls=settings.lookup_rate_limit_max_calls,
    window_seconds=settings.lookup_rate_limit_window_seconds,
)
```

The reasoning is stated directly in the route's own rejection message: each lookup fans out to every configured provider plus, potentially, an OSINT crawler and multiple AI calls, so this is a cost/load control on the single most expensive operation in the platform, not a generic API throttle. Both the call limit and window are configurable settings; the enforcement itself is per-user (keyed on the authenticated user's ID), not per-IP or global, which means it cannot be used to throttle one user's traffic by exhausting a shared bucket.

A second limiter exists on `/auth/login` (added in a later mission-critical-reliability review, v0.2.3): a per-account limiter keyed by email, defaulting to 10 attempts per 60 seconds, both configurable, returning `429` once exceeded -- a real defense against credential-stuffing/brute-force attempts against one specific account. `/auth/register` still has no rate limiting of any kind, and there is no rate limiting anywhere else in the API. This is stated plainly in §12 rather than left implicit.

10. Audit Logging

`config_audit_log` is an append-only table capturing every provider/AI-backend configuration change and every user-management/authentication event, each row carrying a timestamp, an `actor_user_id` (nullable FK) plus a denormalized `actor_email` (so history stays readable even if the account is later disabled), an `action` string, and a free-text `detail` field capped at 1000 characters.

What is captured: configuring, activating, enabling, or disabling an AI backend or IOC provider; testing a provider/AI-backend connection (though see the gap noted below); creating, updating, enabling, disabling, or password-resetting a user account; and every login attempt, successful or failed. A failed login is recorded with the attempted email in `detail` but with `actor_user_id=None` -- there is no authenticated identity to attribute it to -- and the function that records it (`record_login_failure(email: str)`) structurally has no password parameter at all, so there is no code path through which a submitted password could ever end up in this table even by mistake.

What is deliberately never captured. The module's own docstring states the rule directly: `detail` "must be human-readable description text ONLY -- never a credential/password/token value." Every current call site honors this with a hardcoded, credential-free description (for example, `"Configured AI provider '{backend}'."`) -- confirmed directly against a running instance: after exercising a full configure/enable/disable/test/activate cycle, a grep of the audit table's actual contents and the backend logs found no real secret value in either. This is a coding convention enforced by review and by the shape of the functions involved, not by a runtime content filter that would catch a future mistake automatically -- stated here as exactly that, an honest process guarantee rather than a mechanical one.

Read access is admin-only, gated by a dedicated `audit:read` permission independent of `provider:manage` / `user:manage`, so an administrator cannot be locked out of the audit trail by having the wrong *other* permission, and no non-administrator role can read it at all.

Two disclosed, honest gaps: test-connection audit rows (`ai_provider.test`, `ioc_provider.test`) record *what* happened but not *who* triggered it, unlike every other action type in the table, even though the calling route has the authenticated user in scope -- a straightforward fix not yet made. And there is no audit coverage at all for case, basket, or investigation activity -- creating a case, adding a basket item, or running a lookup produces no audit row; only the runtime-configuration and user-management subsystems are audited today. Neither gap is a credential-exposure risk; both are coverage gaps, recorded as such.

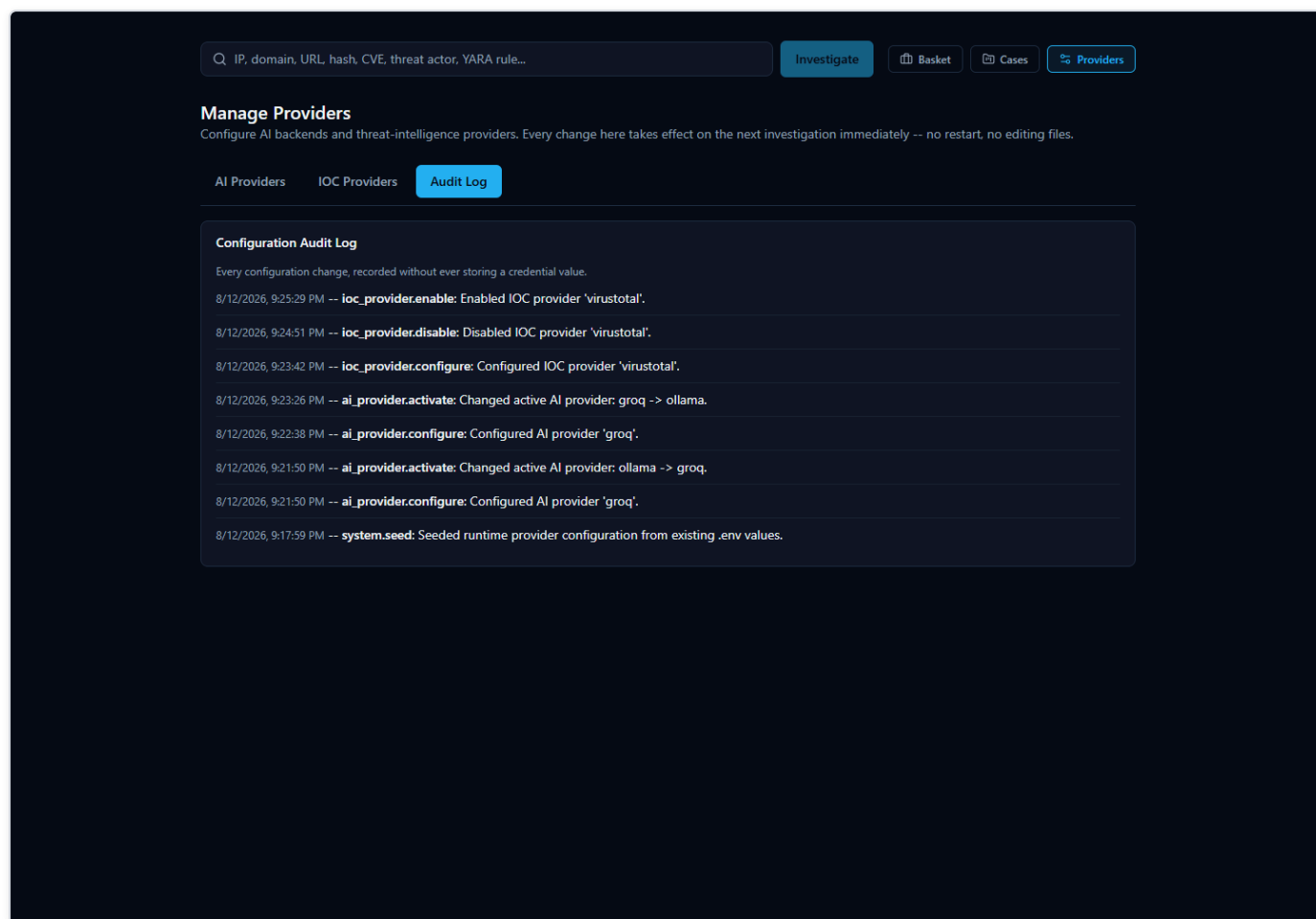


Figure 3 — The Administration console's Audit Log tab, showing account changes and provider-configuration changes together in one chronological timeline, admin-only.

11. CORS and Network Exposure (brief -- see the Security Architecture appendix for full detail)

Browser-originated cross-origin requests are gated by a private-network-shaped CORS regex (matching `localhost`, `127.0.0.1`, and the three RFC 1918 private ranges over `http://` only) rather than a fixed origin list or a wildcard -- this codebase never sets `allow_origins=["*"]`. This is a same-origin-policy relaxation for LAN reachability, not the platform's authorization boundary: a request from a matching origin still needs a valid bearer token for every protected route regardless. Datastore containers (Postgres, Redis, Neo4j, OpenSearch) bind to loopback only; the backend API and frontend web app publish on all host network interfaces by default, restricted at the OS level on Windows installs by a firewall rule scoped to the **Private** network profile only, created at install time and removed on uninstall.

12. Known Limitations and Disclosed Risks

This platform's own release process treats "known limitation, disclosed" as a materially different thing from "silently accepted risk." The following are pulled directly from the most recent release's QA report and the security appendices' own "Recommended (not yet implemented)" lists -- none of these are hidden, and none of them were found to be actively exploited in a running instance:

- **No rate limiting on `/auth/register`, and no account logout on either authentication endpoint.** `/auth/login` itself gained a real per-account rate limiter in a later mission-critical-reliability review (v0.2.3, §1, §9).
- **No self-service session revocation.** A user cannot invalidate their own other sessions without an administrator-driven password reset (§1).
- **No multi-factor authentication and no server-side password complexity rule** beyond an 8-character minimum.
- **The "Test Connection" button never falls back to an already-saved credential** (§3) -- a deliberate security property, not a bug, but confirmed during the most recent release's QA cycle to be a real point of user confusion the first time it's encountered; now documented explicitly for exactly that reason.
- **A non-blocking scoring hardening item remains open:** `_provider_votes()` would accept a `NaN` / `Infinity` value as full-strength "malicious" rather than rejecting it; no live provider currently produces such a value, so there is no active exploit path today, but the guard itself has not yet been added.
- **No key-rotation procedure** exists for `jwt_secret_key` or `encryption_master_key` -- rotating either without a migration step would invalidate all existing sessions and/or make previously encrypted runtime credentials undecryptable.
- **The Fernet encryption key falls back to a value derived from `jwt_secret_key`**, not an independently generated secret, unless an operator explicitly sets `encryption_master_key` -- a documented defense-in-depth tradeoff (§3), not HSM-grade key separation.
- **No audit coverage for case, basket, or lookup activity**, and test-connection audit rows lack actor attribution (§10).
- **No HTTPS/TLS anywhere in this architecture.** The platform is designed for a trusted local network, not a hostile one, with no certificate-provisioning story of its own.
- **A local Ollama-backed executive summary can be slow under heavy concurrent load**, because a single local model serializes generation requests -- an infrastructure/model-concurrency characteristic, not a data-correctness bug, and does not affect the dashboard's core KPI or provider-health surfaces (both pure database reads).

Two positive, verified findings from the same release round out this section honestly: a full cross-cutting RBAC sweep of every mission-touched route came back clean (§2), and a dedicated red-team review found **no SQL injection risk in any query** in the same body of work (every query is parameterized via SQLAlchemy Core), **no auth bypass, no privilege escalation, no cross-user data leak, no credential leak, no command injection, and no database corruption** anywhere in that release's surface. The scoring-engine correlation-flood fix (§8), in particular, has progressed all the way from "found, fixed, unit-tested" to fully live-verified end-to-end on a fresh, real, user-installed instance -- it is not carried in this list as an open item, precisely because it no longer is one.

Summary

Area	Strongest control in place	Most significant disclosed gap
Authentication	JWT + bcrypt; <code>token_version</code> -based instant revocation on password reset/role change/disable; per-account rate limiter on <code>/auth/login</code> (v0.2.3)	No self-service logout-everywhere; no rate limiting on <code>/auth/register</code>
Authorization	Single-source <code>ROLE_PERMISSIONS</code> matrix, one shared dependency, race-safe last-admin protection	No per-case/per-lookup ACL (deliberate, team-shared scope)
Credentials	Fernet-at-rest encryption; masked-only API display, never round-tripped	Default encryption key is derived from <code>jwt_secret_key</code> , not independent, unless set explicitly
SSRF	<code>assert_safe_outbound_url()</code> blocks link-local/metadata addresses on the one fully-operator-chosen outbound URL	Scoped to Ollama's <code>base_url</code> only -- the one surface that needs it today
Exports	CSV formula-injection and PDF markup-injection both fixed and regression-tested; export gated on the correct dedicated permission	--
AI / scoring	Score computed before any AI call, mechanically re-applied to AI output; correlation-flood corroboration discount closes a real found vulnerability, now live-verified end-to-end	The residual, non-blocking <code>NaN / Infinity</code> vote-parsing hardening item (§8)
Rate limiting	Redis fixed-window limiter on the one genuinely expensive endpoint (<code>/lookup/stream</code>)	No coverage on authentication endpoints
Audit logging	Actor-attributed, append-only log of every credential/config/user-management/login event, admin-only read	No case/basket/lookup activity coverage; test-result rows lack actor attribution

None of the items in §12 are release-blocking on their own; they are the concrete, code-verified list a production hardening pass would work through next, stated plainly rather than smoothed into marketing language.